

Advanced Machine Learning

DATA 642

Lecture 8 — Connection to Machine Learning

*Figures and notes are from the paper "Introduction to Tensor Decompositions and their Applications in Machine Learning," (<https://arxiv.org/pdf/1711.10781>)

Tensors and machine learning

Data Representation: Tensors are natural data structures for representing multidimensional data, such as images, videos, and time-series data. In machine learning, these data types are prevalent, and tensors provide an efficient and convenient way to represent them.

Model Parameters: Many machine learning models, such as neural networks, are parameterized by tensors. The weights and biases in these models are often represented as tensors, and the computations involved in training and inference operations rely heavily on tensor operations.

Tensor Decompositions: Tensor decompositions are mathematical techniques used in machine learning for dimensionality reduction, feature extraction, and model interpretation. Decomposing tensors into lower-dimensional components can reveal latent structures and patterns in the data, which is useful for tasks like clustering, anomaly detection, and recommender systems.

Tensor-Based Algorithms: Some machine learning algorithms are specifically designed to operate on tensor data. For example, tensor-based methods are used in tasks such as collaborative filtering, multi-relational data analysis, and latent variable modeling.

Temporal Data

- Tensors provide a straightforward way to model the temporal component of relationships represented as matrices (e.g., user preferences in a recommender system, adjacency matrix of a graph).
- Similar to SVD and non-negative matrix factorization (NMF) for matrices, performing decomposition on temporal tensors allows the detection of latent structure in the data.
- Typical tasks in temporal tensor analysis include discovering patterns, predicting evolution, and spotting anomalies.
- Tensor methods can handle never-ending continuous streams of new data without requiring an infinite time dimension.
- Temporal data introduce another structural constraint to tensors, restricting arbitrary permutations in terms of the tensor's dimensions.
- The temporal interpretation adds an additional relationship between the data points stored in the tensor.

Multi-relational Data

Multirelational Data Representation

- Tensors naturally represent multirelational data, such as social networks, where interactions between users are stored as (subject, relation, object) triplets.
- This multimodal structure is generalized by the notion of multilayer networks, where each slice of the tensor represents one of the underlying relations.
- Tensor factorization algorithms applied in this context allow for determining interdependencies occurring on multiple levels simultaneously.
- For example, tensor factorization enables solving challenging tasks in statistical relational learning, such as collective classification, word representation learning, community detection, and coherent subgraph learning.

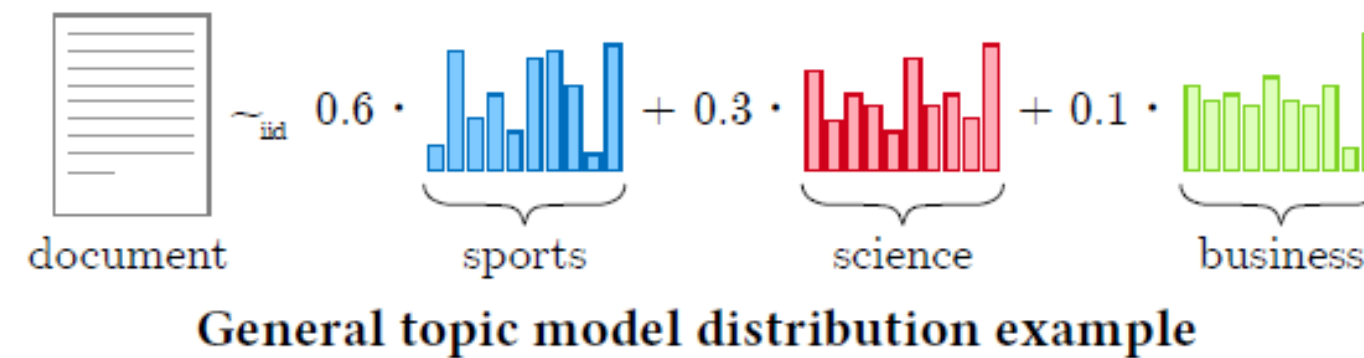
Multi-relational Data (cont.)

Knowledge Networks Analysis

- Knowledge graphs, like Google Knowledge Graph, YAGO, or Microsoft Academic Graph, are special cases of multilayer networks used to store facts about relationships between real-world entities.
- Analyzing knowledge graphs involves inferring new relations between objects based on existing data.
- Tensor decomposition approaches provide state-of-the-art performance in terms of both quality and computational efficiency for analyzing knowledge graphs.
- Information derived from tensor decomposition can be used in applications such as question answering and entity resolution.

Latent Variable Modeling

- Tensor decomposition methods have gained significant traction in machine learning, particularly in inference in latent variable models, over the last decade.
- These methods have been successfully applied to various latent variable models such as hidden Markov models, independent component analysis, and topic models.



- In a typical scenario, a probabilistic model explains how hidden variables generate observed data. Solving the inference problem entails finding the most probable hidden variable settings given observations. Although classic algorithms like maximum likelihood estimation are consistent, they struggle in high dimensions.
- Tensor decomposition methods offer an alternative strategy: they compute empirical data moments (e.g., mean, variance, skewness) and seek latent variable configurations that yield similar quantities within the model.

Current Research Directions

Most current research at the intersection between tensors and machine learning focuses on two main questions:

Formulating Machine Learning Problems as Tensor Decompositions:

- Efforts are underway to explore how various machine learning problems can be formulated as tensor decompositions.
- While some machine learning problems can already be efficiently solved through tensor decompositions, ongoing research aims to determine if tensor methods can also benefit other machine learning algorithms, such as neural networks.

Computing Tensor Decompositions under Weaker Assumptions:

- Tensor decompositions typically have weak conditions for uniqueness, but the requirements for effectively using them in machine learning settings are often stringent.
- For instance, certain applications like Gaussian Mixture Model (GMM) estimation impose strong limitations, such as the requirement that the number of components (k) must be less than or equal to the data dimensionality (d).

Available Software Libraries

Many programming languages provide data structures for multi-dimensional arrays, either as part of their standard libraries or through widely used external packages. Tensors and their decompositions have only recently gained traction in the applied computer science community, so most libraries are primarily available for proprietary environments like Matlab and Python.

Some popular tensor libraries as of 2024 include:

- **TensorLy**: Available for Python.
- **N-way Toolbox**: Available for Matlab.
- **pytensor**: Available for Python.
- **scikit-tensor**: Available for Python.
- **SPLATT**: Available for C/C++, Octave, Matlab.
- **rTensor**: Available for R.
- **Tensor Toolbox**: Available for Matlab.
- **Tensorlab**: Available for Matlab.